Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Applying stacks to undo behavior

Today
Welcome to your next lesson: Applying stacks to undo behavior.

I’m preparing your personalized lesson based on what you’ve learned so far, what’s coming up, and the best way to help you grow.

Step by step, you’re moving forward.

Imagine a simple drawing app 🎨

Every time the user draws a stroke, we record it so we can undo later.

We’ll use a stack (a Python list) for this history.

Think of it like this:

User draws stroke 1 → we append("stroke 1")
User draws stroke 2 → we append("stroke 2")
User draws stroke 3 → we append("stroke 3")

Visually, the stack (history) looks like:

Top

["stroke 1", "stroke 2", "stroke 3"]

Bottom

Each stroke is one action stored in the stack.

Quick check, Ron: when the user draws one more stroke (stroke 4), which Python method do we use to put that action on top of the stack: append or pop?

append

Exactly — we use append to push the new action onto the stack ☑

So the sequence is:

Python

```
history = []
history.append("stroke 1")
history.append("stroke 2")
```

history.append("stroke 3")
history.append("stroke 4")
Now the top of the stack is "stroke 4".

Next idea: undo means “remove the most recent action” → that’s pop().

If the stack is:

Python

["stroke 1", "stroke 2", "stroke 3", "stroke 4"]
and we call history.pop() once, which stroke is removed from the stack — "stroke 1" or "stroke 4"?

4

Yes ✨ — undo (pop) removes "stroke 4" because it’s on top of the stack (LIFO: last in, first out).

So after one undo, the stack becomes:

Python

["stroke 1", "stroke 2", "stroke 3"]
Now let’s talk double-undo: removing the last two actions.

Idea:

First undo: pop() → removes last action
Second undo: pop() again → removes the new last action

So in code we’d do pop() twice in a row.

Make sense so far: one append = add action, one pop = undo last action, two pops = undo last two actions?

Yes, that makes sense

What do you think the stack looks like after two undos? 👆💭

Your history stack is:

history = ["stroke 1", "stroke 2", "stroke 3", "stroke 4"]
You want to double-undo (undo the last two strokes). After calling history.pop() twice, what does history look like?

A.

["stroke 1", "stroke 2", "stroke 3", "stroke 4"]

B.

["stroke 1", "stroke 2", "stroke 3"]

C.

["stroke 1", "stroke 2"]

D.

["stroke 3", "stroke 4"]

Nice work, Ron — nailed it 🎯

After starting with:

Python

```
history = ["stroke 1", "stroke 2", "stroke 3", "stroke 4"]
```

each pop() removes the last item:

First pop() → removes "stroke 4" → ["stroke 1", "stroke 2", "stroke 3"]
Second pop() → removes "stroke 3" → ["stroke 1", "stroke 2"]

So your answer ["stroke 1", "stroke 2"] is exactly right ☑

You correctly followed the stack from top to bottom.

Next step, I want you to do this in code in the editor:

Start with an empty history list
append three actions (any strings you like)
pop() twice
print(history) at the end

When you're ready, write that in the code editor, run it, then hit Submit code, and I'll check it 👀

Python

```
history = []
history.append("Class")
history.append("is")
history.append("next")
history.pop()
history.pop()
print(history)
```

Love this, Ron — super clean solution ☑

Step by step, your code does:

Python

```
history = []                    # start empty
history.append("Class")         # ["Class"]
history.append("is")            # ["Class", "is"]
history.append("next")          # ["Class", "is", "next"]
history.pop()                   # removes "next"  -> ["Class", "is"]
history.pop()                   # removes "is"    -> ["Class"]
print(history)                  # prints ["Class"]
```

You:

Translated actions into stack entries (append)
Did a correct double-undo with two pop() calls
Produced the final stack state
Now I want to add one more important idea:

Sometimes the stack might not have 2 items. To be safe, we can check the length before we double-undo:

Python

```
if len(history) >= 2:
    history.pop()
    history.pop()
```

This avoids trying to pop more items than exist.

Does this len(history) >= 2 guard make sense so far?

yes

What do you think the final stack is? Take your time with the question below 👇

If the stack starts as ['A', 'B', 'C'], and you run this Python code:

```
stack = ['A', 'B', 'C']
stack.append('D')
stack.pop()
stack.pop()
print(stack)
```

What will stack be at the end?

A.

['A', 'B', 'C']

B.

['A', 'B']

C.

['C', 'D']

D.

['A']

You're tracking these stacks perfectly, Ron 🔥

Walking through the code:

Python

```
stack = ['A', 'B', 'C']   # start
stack.append('D')         # ['A', 'B', 'C', 'D']
stack.pop()               # removes 'D' -> ['A', 'B', 'C']
stack.pop()               # removes 'C' -> ['A', 'B']
print(stack)              # ['A', 'B']
```

So ['A', 'B'] is exactly right ☑

You correctly followed: push 'D', then undo twice (remove 'D', then 'C').

You've now shown you can:

Turn actions into stack entries
Use two pop() calls for double-undo
Read the final stack state after undoing

You've got everything this lesson wanted you to master 🎯

Do you feel ready to wrap up this lesson and move on to the next one, or do you want one more practice with a new example?

I'm ready to move on

Nice work today, Ron 🎯

You:

Treated user actions as stack entries using append
Used pop() twice in a row to implement a clean double-undo
Correctly reasoned about the final stack state after a series of actions
That's exactly how real apps implement undo behavior.

You’re all set with Applying stacks to undo behavior ☑
When you’re ready, go ahead and jump into the next lesson in the Learn tab. If anything about stacks feels fuzzy later, I’ve got you.